Portfolio Performance under Machine Learning Prediction and MAD Optimization

Group Member: Yue Wu, Deyi Zhang

Jennie Zhan, Mengyuan Su

2026.03.14

STATISTICS 426

Deep Learning

Instructor: George Michailidis

University of California, Los Angeles

Contents

Abstract. This study constructs a comprehensive framework for multi-asset portfolio optimization that combines advanced machine learning-based return predictions with risk management strategies. The goal is to examine whether machine learning-based return prediction can improve portfolio allocation and whether the inclusion of cryptocurrencies enhances portfolio performance. Using daily Yahoo Finance data from 2018 to 2025, we construct a multi-asset dataset that includes cryptocurrencies, traditional stocks, technology stocks, foreign exchange, and commodities. The prediction target is the 21-day forward return, and the dataset is organized in a rolling framework for model training, validation, and testing.

In the prediction stage, we compare Random Forest as a baseline model with Extreme Gradient Boosting and Long Short-Term Memory as advanced models. The results show that Extreme Gradient Boosting provides the most effective return forecasts, so its predictions are used as inputs in the portfolio optimization stage.

In the second stage, portfolio optimization is conducted using the Mean Absolute Deviation model, and the benchmark portfolio without cryptocurrencies is compared with portfolios that each include one cryptocurrency. The results show that cryptocurrency inclusion is beneficial. The benchmark non-crypto portfolio achieves an annual return of 7.5%, annual volatility of 25.9%, and a Sharpe ratio of 0.411, while the best-performing portfolio, which includes XRP, reaches an annual return of 19.4%, annual volatility of 30.4%, and a Sharpe ratio of 0.735. Relative to the benchmark, this represents an improvement of 11.9 percentage points in annual return and 0.324 in Sharpe ratio.Overall, the findings suggest that machine learning-based return predictions, especially those generated by Extreme Gradient Boosting, can improve portfolio construction, and that selected cryptocurrencies can meaningfully enhance risk-adjusted portfolio performance.

Keywords: Machine Learning, Portfolio Optimization, Cryptocurrency, Return Prediction, Extreme Gradient Boosting, Mean Absolute Deviation, Sharpe Ratio.

1 Introduction

Financial return prediction is a long-standing challenge in quantitative finance. Asset returns are highly noisy, non-stationary, and often influenced by complex nonlinear relationships among economic variables. Traditional linear models frequently struggle to capture these dynamics, which has motivated the use of modern machine learning techniques in financial forecasting. Machine learning models such as Random Forests, Long Short-Term Memory (LSTM) networks, and gradient boosting algorithms are capable of identifying nonlinear patterns and interactions in large datasets, making them attractive tools for predicting asset returns.

In this project, we develop a machine learning framework to forecast future asset returns and apply those predictions to portfolio construction. Specifically, we compare three widely used models for predicting 21-day forward returns, with Random Forest (RF) serving as the baseline model and LSTM and Extreme Gradient Boosting (XGBoost) serving as the advanced models. Each model represents a different modeling philosophy. Random Forest is an ensemble tree-based method that captures nonlinear interactions while reducing overfitting through bagging. LSTM is a recurrent neural network designed to model temporal dependencies in sequential data. XGBoost is a gradient boosting algorithm known for its strong predictive performance and robustness in structured data problems.

While predictive accuracy is important, the ultimate objective of return forecasting is to improve investment decisions. Therefore, the second stage of our framework integrates the predicted returns into a portfolio optimization procedure. We use Mean Absolute Deviation (MAD) portfolio opti-

mization, which minimizes portfolio risk measured by mean absolute deviations of returns while maximizing expected return. Compared with variance-based optimization, MAD models are often more robust to non-normal return distributions and are computationally efficient.

Another important question in modern portfolio construction is whether cryptocurrencies can improve portfolio performance. Cryptocurrencies such as Bitcoin and XRP exhibit return characteristics that differ from traditional assets like equities and commodities. Because of these differences, they may provide diversification benefits when included in a portfolio. To investigate this possibility, we compare optimized portfolios both with and without cryptocurrency assets.

The objective of this study is therefore twofold. First, we evaluate the predictive performance of RF, LSTM, and XGBoost models for forecasting short-term asset returns. Second, we assess whether incorporating these predictions into a MAD optimization framework can produce improved portfolio performance, particularly when cryptocurrency assets are included. The results provide insight into both the effectiveness of machine learning models in financial prediction and the potential role of cryptocurrencies in diversified portfolios.

2 Related Work

Recent work related to this project mainly falls into three directions, machine learning for return prediction, prediction-enhanced portfolio optimization, and diversification using cryptocurrencies.

First, recent studies show that machine learning can be combined with portfolio construction rather than being used only for standalone prediction. Huang et al. [1] study machine learning-driven stock return prediction together with portfolio optimization and show that prediction models can be integrated directly into the allocation process. At the same time, Ai et al. [2] also propose a framework for return ranking prediction and portfolio optimization in the M6 competition, showing that prediction and investment decision-making can be closely linked instead of treated as two separate tasks.

Second, recent literature also suggests that cryptocurrencies may improve portfolio diversification, although the effect depends on how they are incorporated. Han et al. [3] study cryptocurrency factor portfolios and examine their out-of-sample diversification benefits when combined with traditional and machine-learning-enhanced asset allocation strategies. Their results suggest that cryptocurrencies can play a useful diversification role, but the benefit is not uniform across all forms of crypto exposure.

Third, some recent studies focus specifically on combining cryptocurrency forecasting with portfolio optimization. Chen and Zheng [4] develop a framework that links cryptocurrency prediction with portfolio optimization using LSTM and multi-task neural network methods. This line of work is closely related to the current project because it also treats prediction and optimization as connected stages.

Overall, these studies show that recent approaches often combine prediction and portfolio construction, and they also highlight the possible diversification value of cryptocurrencies. Based on these ideas, this project combines return prediction and portfolio optimization in a two-stage framework and examines whether adding a selected cryptocurrency can further improve portfolio performance.

3 Dataset and Exploratory Data Analysis (EDA)

3.1 Data Type

This study uses a multi-asset financial time-series dataset covering the period from 2018 to 2025 at a daily frequency. The dataset includes traditional assets and selected cryptocurrencies, spanning several asset classes such as equities, foreign exchange, commodities, and cryptocurrencies. The assets are shown in Table 1.

Table 1: Asset Universe

Asset Class	Asset Name	Ticker
Cryptocurrencies	Bitcoin	BTC-USD
	Ethereum	ETH-USD
	XRP	XRP-USD
	Litecoin	LTC-USD
	Bitcoin Cash	BCH-USD
Traditional Stocks	Berkshire Hathaway	BRK-B
	Johnson & Johnson	JNJ
	JPMorgan Chase	JPM
	Procter & Gamble	PG
	Visa	V
Tech Stocks	Apple	AAPL
	Amazon	AMZN
	Alphabet	GOOGL
	Meta Platforms	META
	Microsoft	MSFT
Foreign Exchange	Australian Dollar / U.S. Dollar	AUDUSD=X
	Canadian Dollar / U.S. Dollar	CADUSD=X
	Euro / U.S. Dollar	EURUSD=X
	British Pound / U.S. Dollar	GBPUSD=X
	Japanese Yen / U.S. Dollar	JPYUSD=X
Commodities	Gold Futures	GC=F
	Copper Futures	HG=F
	Coffee Futures	KC=F

In addition to asset-level information, this study incorporates the S&P 500 index as a market benchmark. The index itself is not used as a prediction target. Instead, it is used to construct market-level explanatory features, which are merged into the asset-level dataset by date.
The prediction target is the 21-day-ahead return,

$$y_{i,t} = \frac{P_{i,t+H}}{P_{i,t}} - 1, \qquad H = 21 \quad (1),$$

Here $P_{i,t}$ denotes the closing price of asset i at time t. This design makes the forecasting task forward-looking while ensuring that all predictors are constructed using information available at or before time t.

3.2 Notation

Throughout this study, i indexes assets and t indexes time. The one-period return of asset i at time t is denoted by $r_{i,t}$. Conceptually, it can be written as

$$r_{i,t} = \frac{P_{i,t}}{P_{i,t-1}} - 1 \quad (2).$$

Market returns, represented by the S&P 500 index, are denoted by $r_t^{(m)}$. All features are constructed using only historical information up to time t.

3.3 Training, Validation, and Test Splits

This study uses a rolling-window framework to split the dataset into training, validation, and test periods in chronological order, so that the model is always estimated on past data and evaluated on future data.

Specifically, each rolling experiment uses three consecutive years for training, the following one year for validation, and the next one year for testing. For example, in the first rolling window, data from 2018–2020 are used for training, data from 2021 are used for validation, and data from 2022 are used for testing. In the second rolling window, the entire window moves forward by one year, so that 2019–2021 are used for training, 2022 for validation, and 2023 for testing. Thus, the test period covers four years, from 2022 to 2025, through a sequence of rolling out-of-sample evaluations. This chronological split design prevents data leakage and provides a realistic setting for financial forecasting and subsequent portfolio optimization.

3.4 Exploratory Analysis

The exploratory analysis focuses on understanding the structure of the dataset before model fitting, with particular attention to the distribution of the target variable, feature behavior, and differences across asset classes.

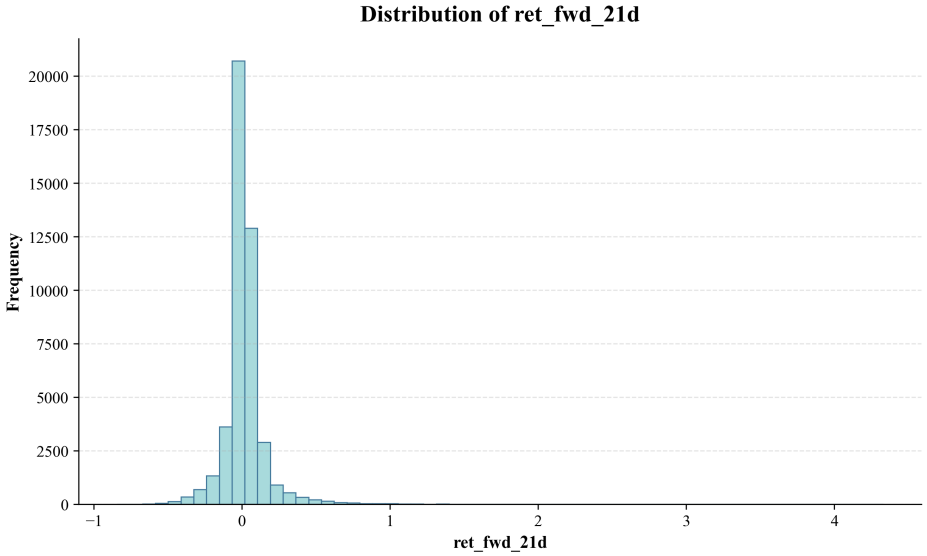


Figure 1: Distribution of the 21-day-ahead return ret_fwd_21d.

Figure 1 shows the distribution of the 21-day-ahead return ret_fwd_21d. Most observations are concentrated around zero, while a smaller number of large positive and negative returns appear in the tails. This indicates that the target variable is continuous, noisy, and slightly heavy-tailed, which is typical in financial return prediction.

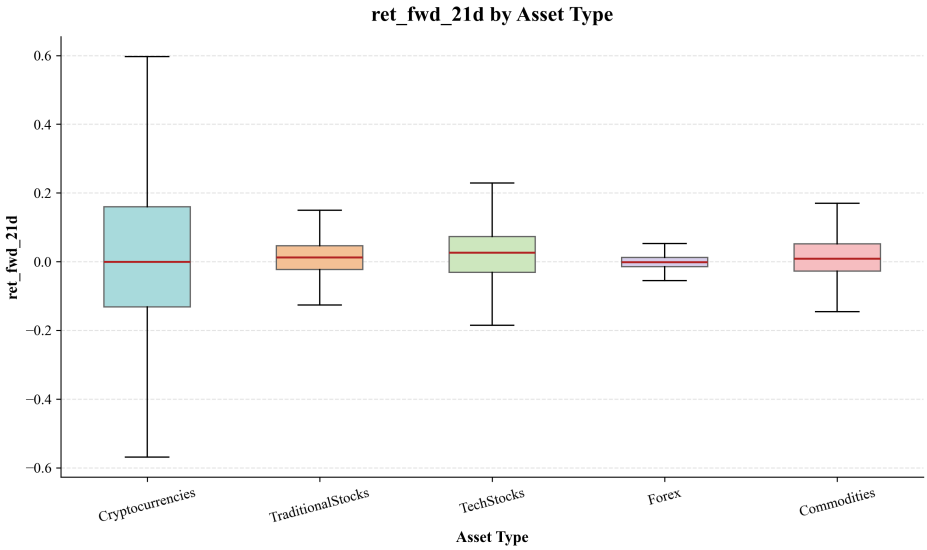


Figure 2: Distribution of ret_fwd_21d across asset classes.

Figure 2 compares the target distribution across asset classes. Clear heterogeneity can be observed. Cryptocurrencies have the widest spread and the highest volatility, while foreign exchange series are much more concentrated around zero. Traditional stocks, technology stocks, and commodities lie between these two extremes. This pattern suggests that the prediction task is more challenging in a multi-asset setting, but also more realistic for later portfolio allocation.

Overall, the exploratory analysis indicates that the dataset contains substantial cross-asset variation and a noisy continuous target. These characteristics motivate the use of flexible nonlinear models, such as LSTM and XGBoost, to capture more complex relationships than simple linear methods.

3.5 Preprocessing

First, all observations with missing return values are removed at the beginning of the process, since return is a core variable required for subsequent feature construction. After this filtering step, all observations are sorted by ticker and date.

Second, the S&P 500 series is extracted from the original dataset. Using its return series, four market-level variables are constructed:

$$\bar{r}_{t,5}^{(m)} = \frac{1}{5}\sum_{k=0}^{4} r_{t-k}^{(m)} \quad (3),$$

$$\sigma_{t,20}^{(m)} = \sqrt{\mathrm{Var}\left(r_{t-19:t}^{(m)}\right)} \quad (4),$$

$$M_{t,20}^{(m)} = \prod_{k=0}^{19}\left(1 + r_{t-k}^{(m)}\right) - 1 \quad (5).$$

Together with the contemporaneous market return $r_t^{(m)}$, these variables form the S&P 500 benchmark feature set. They are merged into the asset-level dataset by date, after which the S&P 500 itself is removed from the prediction target universe.

3.6 Feature Engineering

The final feature space contains 42 predictors for each asset-date observation shown in Table 2.

Table 2: Feature groups used in the final machine learning dataset

Feature Group	Count	Variables
Lagged Return Features	20	`lag1--lag20`
Rolling Statistical Features	6	`mean5`, `vol5`, `mean20`, `vol20`, `mom20`, `range20`
Engineered Features	12	`rsi14`, `atr14_norm`, `vol_ratio`, `skew20`, `price_pos20`, `ema_dev20`, `vol_surge`, `vol_trend`, `sector_mom`, `asset_type_code`, `month`, `day_of_week`
Market Benchmark Features	4	`sp500_ret`, `sp500_mean5`, `sp500_vol20`, `sp500_mom20`

Lagged Returns. To capture short-run temporal dependence, 20 lagged return features are created:

$$r_{i,t}^{(k)} = r_{i,t-k+1}, \qquad k = 1, \ldots, 20 \quad (6).$$

In the implementation, `lag1` corresponds to the return observed at time t, while larger lag indices correspond to progressively older returns.

Statistical Features. Using the lagged returns, several rolling summary statistics are constructed:

$$\bar{r}_{i,t,5} = \frac{1}{5}\sum_{k=1}^{5} r_{i,t-k+1}, \quad (7)$$

$$\sigma_{i,t,5} = \sqrt{\mathrm{Var}(r_{i,t-4:t})}, \quad (8)$$

$$\bar{r}_{i,t,20} = \frac{1}{20}\sum_{k=1}^{20} r_{i,t-k+1}, \quad (9)$$

$$\sigma_{i,t,20} = \sqrt{\mathrm{Var}(r_{i,t-19:t})}, \quad (10)$$

$$M_{i,t,20} = \prod_{k=1}^{20} (1 + r_{i,t-k+1}) - 1, \quad (11)$$

$$R_{i,t,20} = \max(r_{i,t-19:t}) - \min(r_{i,t-19:t}) \quad (12).$$

Technical Indicators. To capture nonlinear price dynamics, the Relative Strength Index (RSI) is defined as

$$RSI_{i,t} = 100 - \frac{100}{1 + RS}, \quad (13)$$

$$RS = \frac{\mathrm{EWMA}(\mathrm{gain}, 14)}{\mathrm{EWMA}(\mathrm{loss}, 14)} \quad (14).$$

The normalized Average True Range (ATR) is defined by

$$ATR_{i,t} = \mathrm{EWMA}(TR_{i,t}, 14), \quad (15)$$

$$ATR_{i,t}^{\text{norm}} = \frac{ATR_{i,t}}{P_{i,t}} \quad (16).$$

In addition, the dataset includes several derived variables related to volatility, asymmetry, and relative price position, such as the volatility ratio (vol5/vol20), rolling skewness over the past 20 returns, the relative price position within a 20-day high–low window, and deviation from the 20-day exponential moving average.

Volume Features. Two variables are used to represent trading activity, the volume surge, measured as current volume relative to its 5-day moving average, and the volume trend, defined as the ratio of the 5-day to 20-day moving average of volume. If volume is missing or zero, these variables are treated as missing before final imputation.

Cross-Sectional Features. To capture within-group co-movement, an asset-type average return is constructed by date:

$$M_t^{(\text{sec})} = \frac{1}{N_s} \sum_{i \in s} r_{i,t}, \quad (17).$$

Here s denotes an asset class and N_s is the number of assets in that group. In the implementation, this quantity is based on the contemporaneously available lagged return measure.

Time Features. Calendar effects are captured using time indicators such as month and day of the week. In addition, asset type is encoded as a categorical variable (using integer encoding for tree-based models). For neural network models, care is taken to avoid introducing artificial ordering effects.

3.7 Final Dataset Construction

The final machine learning dataset includes 20 lagged return features, 6 rolling statistical features, 12 engineered features, 4 S&P 500 market benchmark features, and the target variable $y_{i,t}$. Observations with missing lagged-return features, core rolling statistical features, or target values are removed because these variables are essential for model fitting. For the remaining engineered variables, including technical, volume, cross-sectional, calendar, and market features, any leftover missing values are filled with zero. No explicit data augmentation is applied. In addition, no separate global normalization step is used at this stage; instead, some scale adjustment is handled through ratio- and deviation-based engineered features. This design preserves the original temporal structure of the financial time-series data and maintains consistency in chronological evaluation.

4 Methodology

For the machine learning prediction models, we choose Random Forest (RF) as a baseline model to establish a strong nonlinear benchmark. To further improve predictive performance, we introduce more advanced models, including XGBoost and Long Short-Term Memory (LSTM) networks. These models are selected for their ability to capture complex nonlinear patterns, interactions among predictors, and temporal dependencies in financial data.

The model that demonstrates the best performance is selected and its predicted returns are used as inputs for portfolio construction. In this way, the methodology integrates the prediction stage with the decision-making stage, forming a complete pipeline from return forecasting to portfolio optimization.

4.1 Baseline Model: Random Forest (RF)

We use Random Forest (RF) as the baseline machine learning model for return prediction. RF is an ensemble learning method that combines many decision trees to improve predictive accuracy and reduce overfitting. Instead of relying on a single tree, RF aggregates predictions from multiple trees built on different random subsets of the data.

Let $x_{i,t}$ denote the feature vector used for prediction at time t for asset i, which includes lagged returns (e.g., $r_{i,t-1}, r_{i,t-2}, \dots$), technical indicators, calendar variables, and market-related features. For a given input feature vector $x_{i,t}$, the RF prediction is

$$\hat{y}_{i,t}^{RF} = \frac{1}{B} \sum_{b=1}^{B} T_b(x_{i,t}), \quad (18).$$

Here $T_b(\cdot)$ denotes the prediction from the b-th regression tree and B is the total number of trees. In this study, the RF model uses the lagged returns, technical indicators, calendar variables, asset characteristics, and market-related features introduced as inputs, and predicts the 21-day forward return. The final specification contains 500 trees. To improve generalization, each tree is trained using only a random subset of the training observations, with the subsampling ratio set to 0.8. In addition, at each split, only 40% of the available features are randomly considered. We also set the minimum number of samples in each leaf node to 5, which helps prevent overly deep trees and reduces the risk of overfitting.

The intuition behind RF is straightforward. A single decision tree may fit the training data too closely and produce unstable predictions. By averaging across many decorrelated trees, RF reduces variance and produces a more robust forecast. This is especially useful in financial return prediction, where the relationship between predictors and future returns is often weak, noisy, and nonlinear.

4.2 Advanced Model 1: Extreme Gradient Boosting (XGBoost)

To further improve predictive performance, we use Extreme Gradient Boosting (XGBoost) as an advanced tree-based model. Compared with Random Forest, XGBoost builds trees sequentially rather than independently. Each new tree is trained to correct the prediction errors made by the previous trees, which allows the model to capture more complex nonlinear patterns in the data.

As in the RF model, let $x_{i,t}$ denote the feature vector used for prediction at time t. And given the feature vector $x_{i,t}$, the XGBoost prediction can be written as

$$\hat{y}_{i,t}^{XGB} = \sum_{m=1}^{M} f_m(x_{i,t}), \quad (19).$$

Here $f_m(\cdot)$ denotes the m-th regression tree and M is the total number of boosting rounds. Instead of averaging many independent trees, XGBoost adds trees one by one to gradually improve the fit. At each boosting step, the model minimizes an objective function consisting of a loss term and a regularization term,

$$\mathcal{L} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \Omega(f), \quad (20).$$

Here the first term measures prediction error and the second term penalizes model complexity. This regularization helps control overfitting and improves out-of-sample generalization.

In this study, XGBoost uses the same input features as the other models, including lagged returns, technical indicators, calendar variables, asset characteristics, and market-related features, to predict the 21-day forward return. The final model is implemented with 2000 boosting rounds, a learning

rate of 0.02, and a maximum tree depth of 4. To improve robustness, each tree is trained using 70% of the training observations and 60% of the available features. In addition, the model sets the minimum child weight to 5, the L1 regularization parameter to 0.1, and the L2 regularization parameter to 2.0.

To avoid overfitting, early stopping is applied based on validation-set performance, with the training process stopped if validation RMSE does not improve for 50 consecutive rounds. The final XGBoost model therefore combines boosting, shrinkage, subsampling, and regularization to produce stable and accurate return forecasts.

Compared with RF, XGBoost is more adaptive because it learns from previous prediction errors at each stage. This makes it a strong candidate for financial return prediction, where the underlying relationships are often nonlinear, noisy, and difficult to capture with a single model.

4.3 Advanced Model 2: Long Short-Term Memory (LSTM)

To better capture temporal dependence in financial returns, we use a Long Short-Term Memory (LSTM) network as an advanced prediction model. Unlike tree-based models, LSTM is designed for sequential data and is able to learn patterns from ordered historical observations. This makes it suitable for return prediction, where recent price movements may contain useful time-dependent information.

In our model, the input is divided into two parts. The first part is a sequential input consisting of the past 20 lagged returns,

$$x_t^{(seq)} = (r_{t-20}, r_{t-19}, \ldots, r_{t-1}), \quad (21)$$

which is reshaped as a time series and fed into the LSTM layers. The second part is a static input vector containing additional features such as rolling mean, volatility, momentum, RSI, ATR, calendar variables, asset type, and market-related variables. Denote this vector by

$$x_t^{(stat)} \quad (22).$$

The sequential branch of the model contains two stacked LSTM layers. The first LSTM layer has 64 hidden units and returns the full hidden sequence. The second LSTM layer has 32 hidden units and outputs the final sequence representation. A dropout layer is then applied to reduce overfitting. For the static branch, the additional features are passed through a dense layer with 32 units and ReLU activation, followed by batch normalization. The outputs from the sequential branch and the static branch are then concatenated and passed through two fully connected layers with 64 and 32 units, respectively. Finally, a linear output layer produces the predicted 21-day forward return,

$$\hat{y}_t = f\big(x_t^{(seq)}, x_t^{(stat)}\big) \quad (23).$$

The model is trained by minimizing the mean squared error (MSE) loss,

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \quad (24).$$

Here y_i is the realized forward return and $\hat{y}_i$ is the model prediction. We use the Adam optimizer with learning rate 5×10^{-4}, batch size 64, and a maximum of 100 epochs. Early stopping is applied based on validation loss with patience set to 10, and the best model weights are restored to improve out-of-sample generalization.

Before training, both the sequential input and the static features are standardized using the training

set only, and the same transformation is applied to validation and test sets. This prevents information leakage and ensures that the model is trained in a consistent rolling-window framework. Overall, the LSTM model extends the baseline methods by explicitly modeling time dependence in lagged returns while also incorporating cross-sectional and market-related information through the static feature branch. This hybrid design allows the model to learn both sequential dynamics and non-sequential characteristics of asset returns.

4.4 Evaluation Metrics for Machine Learning Models

To evaluate the predictive performance of the machine learning models, we use three main metrics: mean absolute error (MAE), root mean squared error (RMSE), and directional accuracy (DirAcc). These metrics are computed on both the validation set and the test set under the rolling-window framework.

Let $y_{i,t}$ denote the realized forward return and let $\hat{y}_{i,t}$ denote the predicted forward return for asset i at time t. For a set of n observations, the mean absolute error is defined as

$$\text{MAE} = \frac{1}{n}\sum_{j=1}^{n}|y_j - \hat{y}_j| \quad (25).$$

The root mean squared error is defined as

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{j=1}^{n}(y_j - \hat{y}_j)^2} \quad (26).$$

In addition to error-based metrics, we also evaluate whether the model correctly predicts the direction of returns. Directional accuracy is defined as

$$\text{DirAcc} = \frac{1}{n}\sum_{j=1}^{n}\mathbf{1}(\text{sign}(y_j) = \text{sign}(\hat{y}_j)) \quad (27),$$

Here $\mathbf{1}(\cdot)$ is the indicator function. This metric measures the proportion of observations for which the predicted return and the realized return have the same sign. In financial applications, directional accuracy is important because correctly predicting the sign of returns can be more relevant for investment decisions than minimizing numerical forecast error alone.

4.5 Portfolio Optimization Based on XGBoost Predictions

Among the three models, XGBoost performs the best, so we select XGBoost as the final forecasting model for portfolio construction. Therefore, the predicted returns generated by XGBoost are used as the expected return inputs in the portfolio optimization stage.

Let $\hat{\mu}_t = (\hat{\mu}_{1,t}, \ldots, \hat{\mu}_{n,t})^\top$ denote the vector of predicted returns at rebalancing date t, obtained from the XGBoost model. Based on these forecasts, we construct portfolios using the mean absolute deviation (MAD) framework, which is less sensitive to extreme observations than mean–variance optimization.

Let $w_t = (w_{1,t}, \ldots, w_{n,t})^\top$ denote the portfolio weights, and let $R \in \mathbb{R}^{L\times n}$ be the matrix of historical returns over a lookback window of length L. The optimization problem is

$$\max_{w_t} \quad \hat{\mu}_t^\top w_t - \lambda \cdot \text{MAD}(w_t), \quad (28),$$

Here λ is the risk-aversion parameter.

We reformulated it to the following linear program:

$$\min_{w_t,\, z_1,\ldots,z_L} -\hat{\mu}_t^\top w_t + \frac{\lambda}{L}\sum_{\ell=1}^{L} z_\ell, \quad (29),$$

subject to

$$z_\ell \geq (R_\ell - \bar{R})^\top w_t, \quad z_\ell \geq -(R_\ell - \bar{R})^\top w_t, \quad \ell = 1,\ldots,L, \quad (30),$$

$$\sum_{i=1}^{n} w_{i,t} = 1, \quad w_{i,t} \geq w_{\min}, \quad i = 1,\ldots,n, \quad (31).$$

together with asset-specific upper bounds when applicable. Here, z_ℓ are auxiliary variables and $\bar{R}$ is the vector of mean historical returns.

In our implementation, the optimization uses a rolling lookback window of 252 trading days, corresponding to approximately one trading year. The minimum portfolio weight is set to $w_{\min} = 0.02$ for each asset, and the risk-aversion parameter is fixed at $\lambda = 0.10$. In portfolios that include cryptocurrency, an additional upper bound is imposed so that the single crypto asset can account for at most 25% of the total portfolio weight. This constraint is introduced to control concentration risk from the relatively volatile crypto component.

The backtest is conducted from January 2022 to December 2025 with monthly rebalancing. At each month-end rebalancing date, the model first extracts the XGBoost-predicted returns, then estimates the optimal portfolio weights using the previous 252 trading days of realized returns, and finally holds the portfolio until the next rebalancing date. This procedure preserves the chronological order of information and avoids look-ahead bias.

4.6 Evaluation Metrics for Portfolio Optimization

To evaluate the portfolio performance, we report annualized return, annualized volatility, and the Sharpe ratio. Let $r_{p,d}$ denote the daily portfolio return on day d, and let N denote the total number of trading days in the backtest period. The annualized return is computed as

$$\text{Annual Return} = \left(\prod_{d=1}^{N}(1 + r_{p,d})\right)^{252/N} - 1 \quad (32).$$

The annualized volatility is

$$\text{Annual Volatility} = \sqrt{252}\,\text{sd}(r_{p,d}), \quad (33).$$

and the Sharpe ratio is

$$\text{Sharpe Ratio} = \frac{\bar{r}_p}{\text{sd}(r_{p,d})}\sqrt{252}, \quad (34).$$

Here $\bar{r}_p$ is the average daily portfolio return.

We apply this optimization procedure to a benchmark portfolio containing only non-crypto assets and then to a set of augmented portfolios that each add one cryptocurrency. By comparing their annualized return, volatility, and Sharpe ratio, we assess whether including a crypto asset can improve the overall risk-return tradeoff of the portfolio.

5 Experimental Setup

We summarize the computational environment, training cost, and hyperparameter settings for Reproducibility.

5.1 Hardware & Software

Table 3: Hardware and Software Configuration

Component	Specification
CPU	Apple M4 (arm64)
GPU	Not used
RAM	16 GB
OS	macOS
Language	Python 3
Libraries	scikit-learn, XGBoost, PyTorch, NumPy, Pandas

5.2 Compute Time

Table 4: Training Time Comparison

Metric	RF	XGB	LSTM
Avg time per fit (s)	0.33	0.04	10.48
Total time (s)	30.23	3.68	964.57
CPU hours	0.0084	0.0010	0.2679

XGBoost is the most computationally efficient model, while LSTM requires significantly more time due to iterative training.

5.3 Hyperparameters

Table 5: Main Hyperparameters

Parameter	RF	XGB	LSTM
Trees / Epochs	500	2000	100
Learning Rate	–	0.02	5×10^{-4}
Max Depth	–	4	–
Subsample	–	0.7	–
Feature Sample	–	0.6	–
Batch Size	–	–	64
Dropout	–	–	0.2
Optimizer	–	–	Adam
Early Stopping	–	50	10

6 Results

6.1 Machine Learning Prediction Results

After introducing the data preparation process and model design in the previous section, we now present the results of the machine learning prediction stage. The purpose of this part is to compare

the prediction performance of different models and to determine which model is the most suitable for portfolio optimization. In this study, we compare two advanced machine learning models, LSTM and XGBoost, and one Random Forest baseline model. Their performance is evaluated in both the validation set and the test set using three metrics: mean absolute error (MAE), root mean squared error (RMSE), and directional accuracy. MAE and RMSE are used to measure prediction error, where lower values indicate better performance, while directional accuracy measures whether the model correctly predicts the sign of future returns, where a higher value indicates better practical usefulness for investment decisions.

Table 6: Validation and test performance comparison across machine learning models

Model	Val MAE	Test MAE	Val RMSE	Test RMSE	Val DirAcc	Test DirAcc
LSTM	0.0810	0.0793	0.1081	0.1049	0.5506	0.5238
RF	0.0890	0.0847	0.1165	0.1088	0.5157	0.5135
XGB	0.0769	0.0741	0.1029	0.0974	0.5683	0.5374

Table 6 summarizes model performance across MAE, RMSE, and directional accuracy. XGBoost achieves the best results with the lowest validation MAE of 0.0769, the lowest test MAE of 0.0741, the highest directional accuracy of 0.5683 on the validation set, and the highest directional accuracy of 0.5374 on the test set. LSTM follows with slightly higher error metrics, while Random Forest performs the worst across all measures. This overall ranking establishes XGBoost as the strongest predictive model in this study.

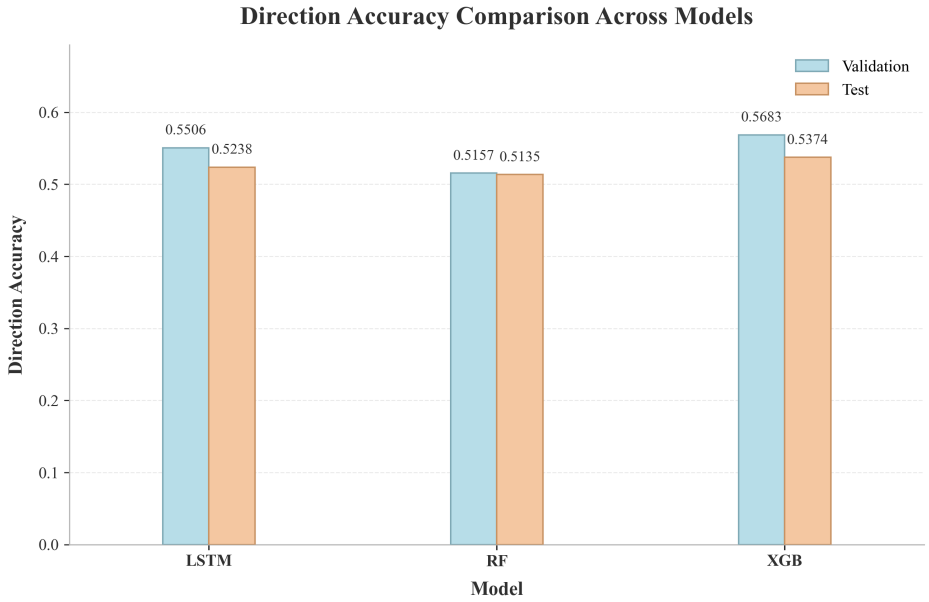


Figure 3: directional accuracy comparison across models on the validation and test sets.

Figure 3 illustrates the comparison of directional accuracy across the three models. This metric is particularly important in financial prediction, as correctly identifying whether returns move upward or downward is often as valuable as estimating their exact values. XGBoost achieves the highest directional accuracy on both the validation and test sets, reaching 0.5683 and 0.5374, respectively. LSTM performs slightly worse, with values of 0.5506 and 0.5238, while Random Forest records the lowest accuracy at 0.5157 and 0.5135. Although the differences are moderate in magnitude, the consistent advantage of XGBoost across both datasets suggests a stronger ability to capture

directional market signals.

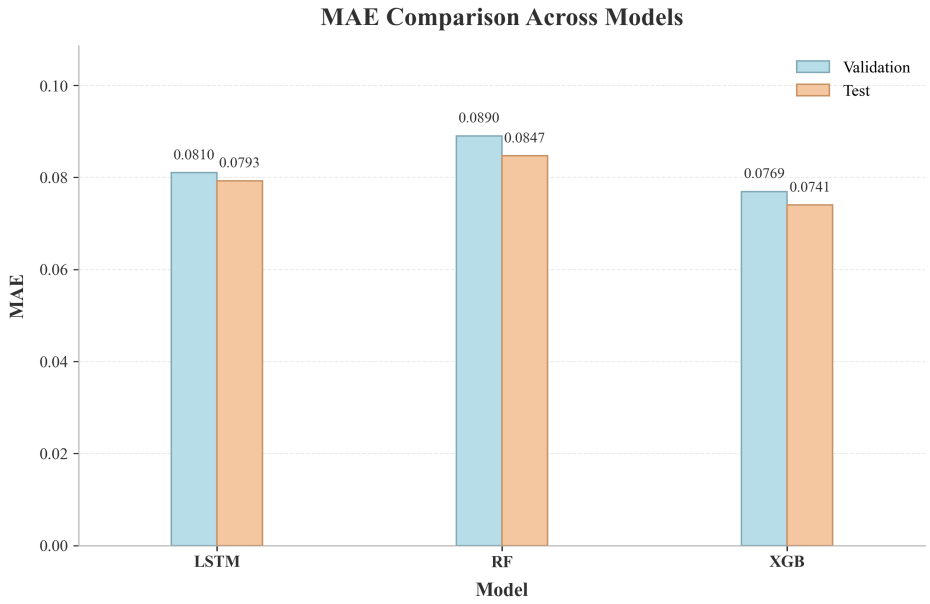


Figure 4: Mean absolute error comparison across models on the validation and test sets.

Figure 4 reports the mean absolute error (MAE) for the three models. XGBoost achieves the lowest MAE on both the validation and test sets, indicating the most accurate predictions in terms of absolute deviation from realized returns. LSTM shows intermediate performance, while Random Forest produces the largest errors. This pattern suggests that XGBoost provides more precise estimates of return magnitude, which is particularly important because portfolio optimization relies directly on predicted returns as inputs.

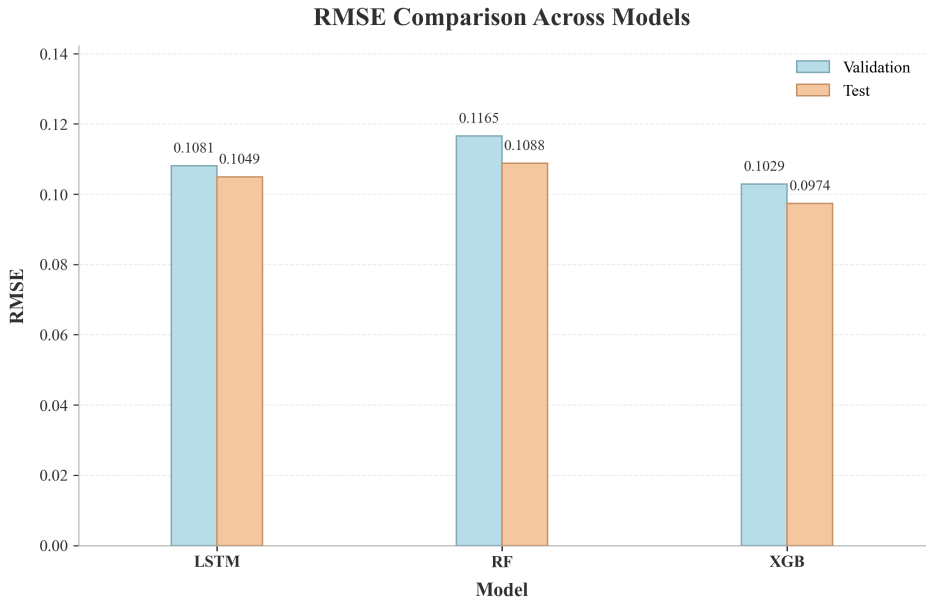


Figure 5: Root mean squared error comparison across models on the validation and test sets.

Figure 5 further evaluates model performance using root mean squared error (RMSE), which places

greater emphasis on larger prediction errors. XGBoost again achieves the lowest RMSE, with values of 0.1029 on the validation set and 0.0974 on the test set, followed by LSTM and then Random Forest. This indicates that XGBoost is more effective at limiting extreme prediction errors in addition to reducing average error. Such stability is especially important in financial applications, where large deviations can lead to poor allocation decisions and increased portfolio risk.
In conclusion, Table 6 and Figures 3–5 consistently show that XGBoost achieves the best performance across all evaluation metrics, while LSTM improves over the Random Forest baseline. XGBoost attains the lowest MAE and RMSE and the highest directional accuracy on both the validation and test sets, indicating its effectiveness in capturing predictive signals. Based on these results, XGBoost is selected as the final prediction model, and its forecasts are used as inputs for the portfolio optimization stage.

6.2 Learning Curve Analysis

To further examine the training dynamics and generalization behavior of the two advanced models, Figures 6 and 7 present their learning curves on the training and validation sets. These curves provide additional evidence on model convergence and help identify potential signs of underfitting or overfitting.

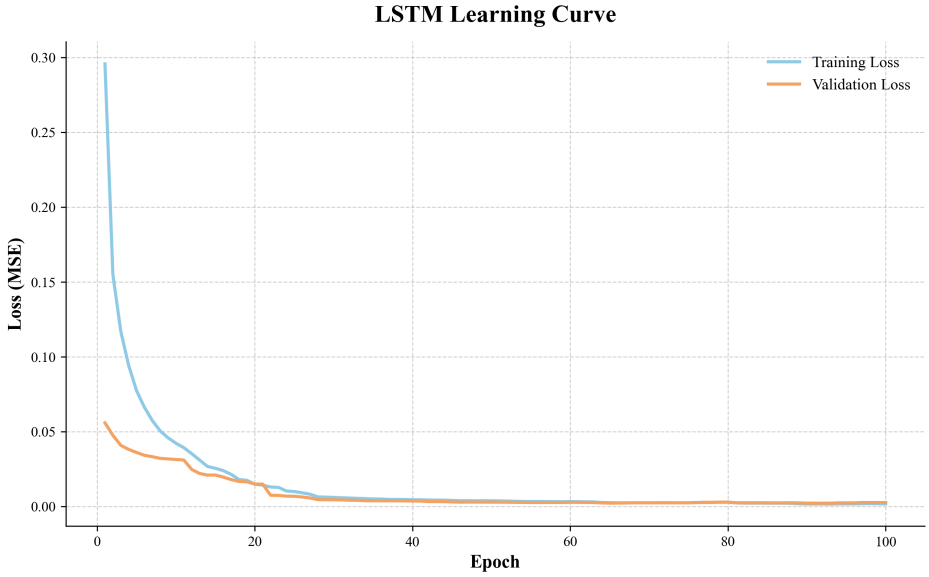


Figure 6: Learning curve of the Long Short-Term Memory model on the training and validation sets.

Figure 6 displays the learning curve of the LSTM. Both the training loss and validation loss decrease rapidly during the early epochs, indicating that the model is able to learn useful patterns from the data at the beginning of training. As the number of epochs increases, the two curves gradually flatten and converge to very low loss values. More importantly, the gap between the training and validation curves remains small throughout most of the training process, and the validation loss does not show a sustained upward trend in later epochs. This suggests that the LSTM does not suffer from severe overfitting. At the same time, since both losses decrease to low levels rather than staying high, there is also no clear sign of underfitting. Overall, the learning curve indicates stable convergence and reasonably good generalization performance.

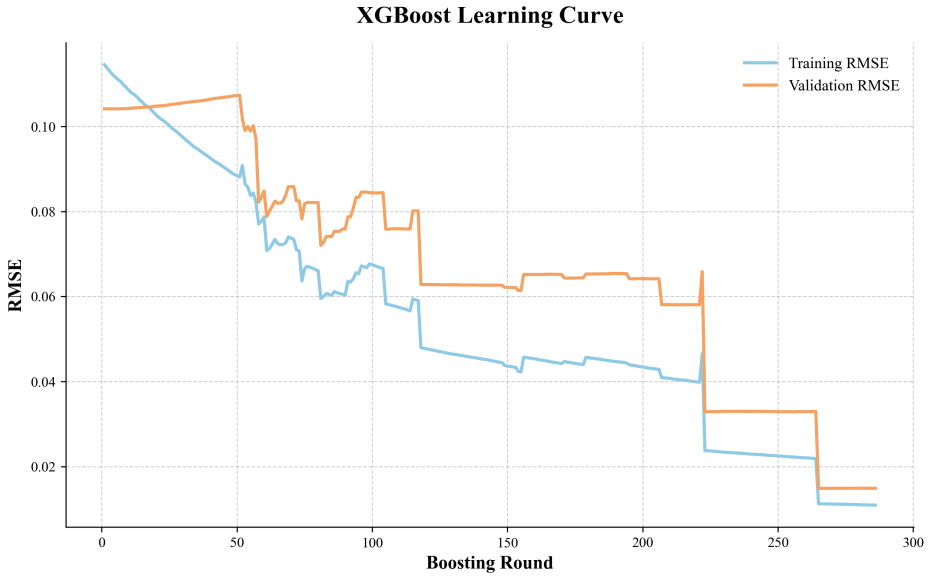
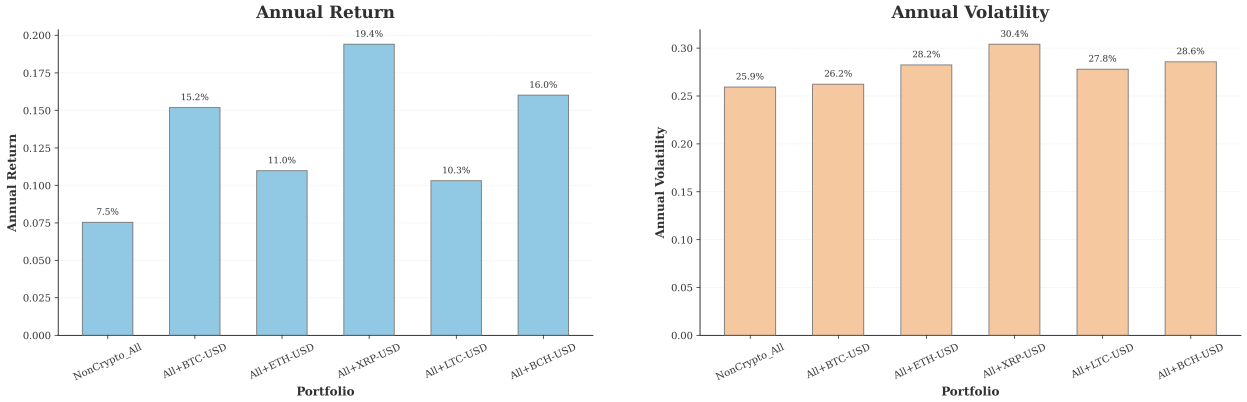


Figure 7: Learning curve of the Extreme Gradient Boosting model on the training and validation sets.

And the Figure 7 presents the learning curve of the XGBoost. The training RMSE declines steadily as the boosting rounds increase, showing that the model continuously improves its fit to the training data. The validation RMSE also decreases. Importantly, the validation curve does not diverge persistently from the training curve. Instead, it continues to decline and reaches its lowest level near the end of training. This indicates that, despite some intermediate variation, the model does not exhibit strong overfitting. In addition, because both the training and validation errors fall substantially over time, there is no evidence of underfitting. The overall pattern suggests that the XGBoost achieves effective learning and strong generalization.

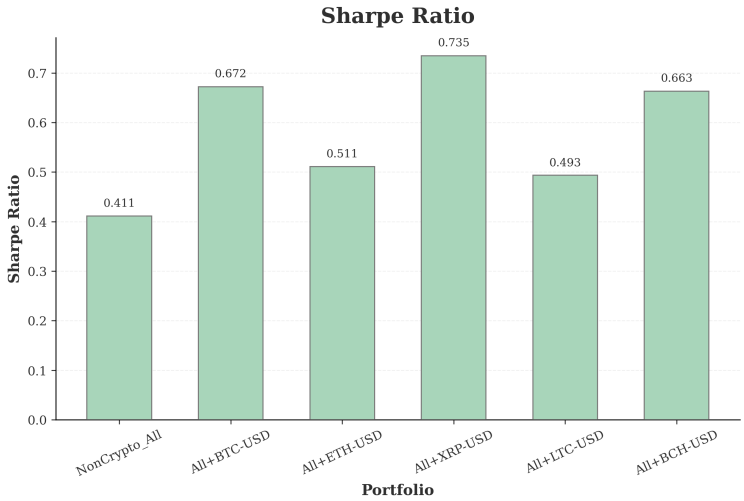
6.3 Portfolio Optimization Results

After selecting XGBoost as the final return prediction model, its predicted returns are used as inputs for the portfolio optimization stage. In this part, the Mean Absolute Deviation (MAD) model is applied to compare the benchmark portfolio without cryptocurrencies with portfolios that each include one cryptocurrency. To keep the allocation practical and avoid excessive concentration, the minimum asset weight is set to 2%, and the weight of the added cryptocurrency is capped at 25%.



(a) Annual return

(b) Annual volatility



(c) Sharpe ratio

Figure 8: Comparison of portfolio performance under MAD optimization.

Figure 8 summarizes the portfolio performance in terms of annual return, annual volatility, and Sharpe ratio. The benchmark portfolio without cryptocurrencies achieves an annual return of 7.5%, annual volatility of 25.9%, and a Sharpe ratio of 0.411. After adding cryptocurrencies, all portfolios show higher annual returns and higher Sharpe ratios than the benchmark, indicating that cryptocurrency inclusion improves overall performance.

Among all candidate portfolios, the portfolio including XRP performs best. It achieves an annual return of 19.4%, annual volatility of 30.4%, and a Sharpe ratio of 0.735. Compared with the benchmark, this represents an increase of 11.9 percentage points in annual return and an improvement of 0.324 in Sharpe ratio. The portfolios including BTC and BCH also perform strongly, with Sharpe ratios of 0.672 and 0.663, respectively. By contrast, the portfolios including ETH and LTC still outperform the benchmark, but their improvements are smaller. Overall, the results show that adding cryptocurrencies is beneficial, although the degree of improvement depends on the specific asset.

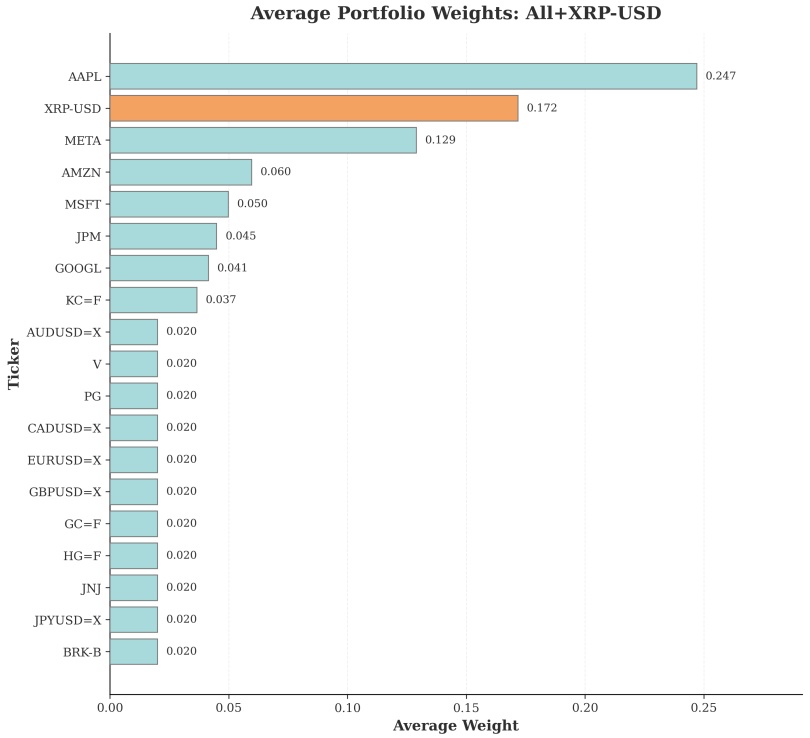


Figure 9: Average portfolio weights of the best-performing portfolio with XRP.

Figure 9 demonstrates the average weights of the best-performing portfolio of XRP. The allocation remains diversified, but more weight is assigned to assets with stronger expected performance. AAPL has the largest average weight at 24.7%, followed by XRP at 17.2% and META at 12.9%. Several other assets, such as AMZN, MSFT, JPM, and GOOGL, also receive moderate allocations, while many remaining assets stay at or near the 2% lower bound. This indicates that the optimizer preserves diversification while concentrating more capital in the most attractive assets.

Overall, these results indicate that combining XGBoost-based return forecasts with MAD optimization improves portfolio performance. In particular, the portfolio including XRP achieves the strongest enhancement in both return and risk-adjusted performance under the given allocation constraints. However, cryptocurrency inclusion also results in higher annual volatility relative to the benchmark portfolio, suggesting that the improved performance is accompanied by increased risk exposure.

7 Discussion & Ablations

7.1 Interpretation of Results

The results are generally consistent with the expectations of this study. In the first part, the prediction stage, both LSTM and XGBoost perform better than the baseline RF across all main metrics. This suggests that in financial prediction tasks more advanced models are better able to capture the nonlinear patterns. Among them, XGBoost gives the best overall results, which means it is the most effective model for this return prediction task. The superior performance of XGBoost is likely due to its ability to model complex nonlinear relationships and interactions in structured financial data. In contrast, LSTM models are better suited for capturing long temporal dependencies, which may be less pronounced in the relatively short historical windows used in this study.

The portfolio optimization results also support the main idea that portfolios with cryptocurrencies

perform better than the benchmark portfolio. In particular, all cryptocurrency-enhanced portfolios achieve higher annual returns and higher Sharpe ratios than the benchmark. This indicates that adding cryptocurrencies can improve portfolio performance under the current framework.
At the same time, the results also show a clear trade-off between return and risk. The portfolios with cryptocurrencies do not only produce higher returns, but also higher volatility. This means the improvement does not come for free. For example, the portfolio including XRP achieves the highest annual return and the highest Sharpe ratio, but it also has the highest volatility. Therefore, the findings should be understood as a risk-return trade-off rather than a pure gain in performance.

7.2 Ablation-Style Comparison

This study treats these comparisons as ablation experiments to isolate the contribution of key components in the pipeline.
First, the choice of model clearly matters. RF gives the basic prediction results, while LSTM and XGBoost both improve prediction accuracy and directional accuracy. This indicates that using a stronger prediction model is an important source of improvement.
Second, not all advanced models contribute equally. LSTM performs well and is clearly better than RF, but XGBoost still performs best on MAE, RMSE, and directional accuracy. This means that, for this dataset and feature design, XGBoost is better suited to extracting useful predictive signals than LSTM.
Third, incorporating cryptocurrency into portfolio optimization is also important. The benchmark portfolio without cryptocurrencies has the lowest return and the lowest Sharpe ratio, while every portfolio with one added cryptocurrency performs better. This suggests that adding cryptocurrencies improves diversification and helps the optimizer build stronger portfolios.
However, the contribution is not the same for every cryptocurrency. XRP gives the strongest improvement, while BTC and BCH also perform well. ETH and LTC still improve the benchmark, but the gains are smaller. So the results suggest that portfolio improvement does not come from simply adding any cryptocurrency. Instead, it depends on which cryptocurrency is added and how well it works with the rest of the portfolio under the MAD framework.
The weight results provide another useful insight. In the best-performing portfolio, the optimizer does not place all the weight on XRP. Instead, it still keeps a diversified allocation and combines XRP with several strong equity positions such as AAPL and META. This shows that the final improvement comes from the interaction between the return forecasts, the optimization model, and cross-asset diversification, rather than from a single asset alone.

8 Conclusion & Future Work

8.1 Main Findings

This study proposes a two-stage framework that combines machine learning-based return prediction with MAD portfolio optimization. The results from both stages provide clear evidence that machine learning-based return prediction can improve portfolio construction, and that adding selected cryptocurrencies can further enhance portfolio performance. In particular, the portfolio including XRP achieves the best overall result, with the highest annual return and Sharpe ratio among all candidate portfolios. At the same time, all cryptocurrency-enhanced portfolios outperform the benchmark in both return and risk-adjusted performance. These findings suggest that the proposed framework is effective in improving allocation outcomes under the current setting.
Moreover, the results imply that portfolio choice may depend on investor risk preference. Investors with a stronger tolerance for risk may prefer the portfolio including XRP because it offers the highest return potential, while more conservative investors may consider alternatives such as BTC

or BCH, which still provide strong performance improvement with relatively lower volatility. While the inclusion of cryptocurrencies improves portfolio returns, it also introduces higher volatility. Additionally, these results are based on a limited test period, and the observed performance may not generalize across different market regimes. Therefore, caution is required when interpreting these results for real-world deployment.

8.2 Limitations

Even though the results are encouraging, the models still have several limitations. First, return prediction is naturally difficult because financial markets are noisy and affected by many outside factors. Even the best model in this study does not achieve extremely high directional accuracy, which means there is still a large amount of uncertainty in the forecasts. In other words, the models improve prediction quality, but they cannot remove the randomness of the market.

Second, the results are not equally strong across all cryptocurrencies. Some assets, such as XRP and BTC, provide much stronger improvements than others. This suggests that the benefit of cryptocurrency inclusion is not fully stable and may depend on the specific asset, its volatility, and its relationship with the other assets in the portfolio.

Another limitation is that the asset universe in this study is relatively selective. Both the traditional assets and the cryptocurrencies are chosen from well-known and widely followed names. This makes the dataset easier to construct and interpret, but it also limits the breadth of the analysis. Less well-known assets or smaller cryptocurrencies may have different risk-return characteristics and diversification effects. Therefore, the conclusions of this study should be understood as applying mainly to a representative set of major assets rather than the entire market.

Overall, the discussion reveals that the main findings of this study do align with expectations. Stronger prediction models lead to better forecast quality, and combining XGBoost-based forecasts with MAD optimization improves portfolio performance. At the same time, the gains are accompanied by higher risk, and the results depend on model choice, asset selection, market conditions, and portfolio constraints.

8.3 Future Work

With an additional six months and substantially greater computational resources, this study could be expanded in several directions. On the prediction side, more advanced models could be tested, such as Transformer-based time-series models, hybrid deep learning architectures, or model ensembles. A broader set of predictors could also be introduced, including macroeconomic variables, sentiment indicators, and more detailed technical features. On the portfolio side, the optimization framework could be extended to include transaction costs, turnover penalties, downside-risk measures, and robustness tests under alternative constraints.

If this project were to be developed into a Master's thesis, several further improvements would be necessary. The machine learning stage could be expanded to include broader model comparisons, systematic hyperparameter tuning, and stronger benchmarking against traditional financial forecasting methods. From a data perspective, the asset universe should be enlarged beyond a set of major assets, and the study should include a longer time horizon and richer explanatory variables. Overall, this project provides a solid foundation for future thesis-level research.

References

[1] Meiyu Huang, Shili Dang, and Miraj Ahmed Bhuiyan, "Multi-objective portfolio optimization for stock return prediction using machine learning," *Expert Systems with Applications*, vol. 298, p. 129672, 2026. DOI: 10.1016/j.eswa.2025.129672.

[2] Hongfeng Ai, Chenning Liu, and Peng Lin, "Robust returns ranking prediction and portfolio optimization for M6," *International Journal of Forecasting*, vol. 41, no. 4, pp. 1494–1504, 2025. DOI: 10.1016/j.ijforecast.2024.04.004.

[3] Weihao Han, David Newton, Emmanouil Platanakis, Haoran Wu, and Libo Xiao, "The diversification benefits of cryptocurrency factor portfolios: Are they there?" *Review of Quantitative Finance and Accounting*, vol. 63, no. 2, pp. 469–518, 2024. DOI: 10.1007/s11156-024-01260-w.

[4] Weiqi Chen and Hang Zheng, "Cryptocurrency price prediction and portfolio optimization based on LSTM and multi-task neural network," *Quantitative Finance and Economics*, vol. 9, no. 3, pp. 658–681, 2025. DOI: 10.3934/QFE.2025023.

Appendix

Author Contributions

Yue Wu led project design, machine learning modeling, and portfolio optimization; Deyi Zhang handled data collection, preprocessing, and feature engineering; Mengyuan Su conducted empirical analysis, visualization, and results interpretation; and Jennie Zhan contributed to the literature review, data preprocessing support, results interpretation, manuscript writing and editing, and presentation organization.

Code Availability

The code for this project is available at:
GitHub Repository